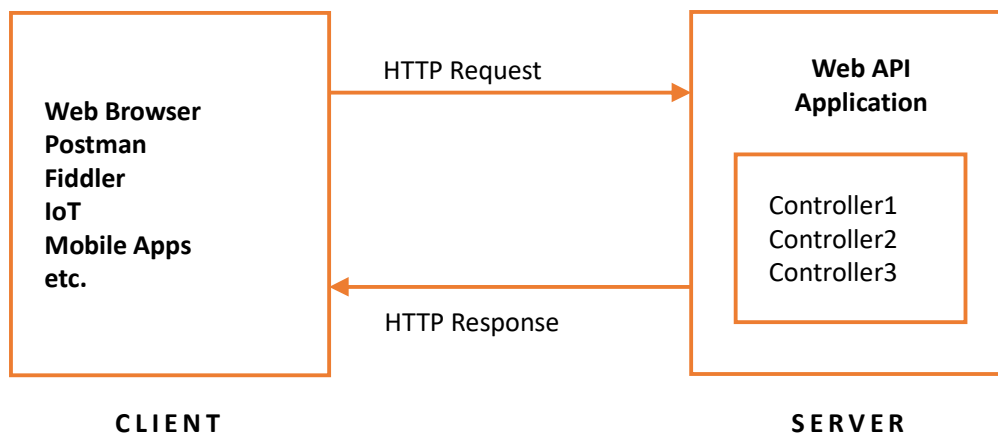


Web API - Routing



Was ist Routing in der WebAPI?

Routing ist das Konzept der Zuordnung zwischen URL und Resource (controller action method), d. h. den HTTP Request auf die entsprechende Controller Methode zu mappen.



Wie funktioniert Routing in der WebAPI?

- **Routing-Configuration** (Conventional Routing) in Program.cs mittels Routing-Middleware für die Request Processing Pipeline.
- **Route Templates** (Attribute Routing) mittels Route-Attributes wie [HttpGet], [HttpPost], [HttpPut], [HttpDelete], [Route] usw. gilt als flexibler und weiter verbreitet.
- **Route Matching** anhand des Parsen der URL durch die Routing Middleware. Das enthält auch das **Parameter Binding** und Konvertierung der URL-Segmente an die Parameter der Methode.
- **Routing Constraints** um beispielsweise den Datentyp von Parametern einzuschränken.

Routing – Mapping vs. Attributes

Konventionelles Routing

Pro

- Zentralisierte Konfiguration
- Konsistenz

Contra

- Weniger flexibel
- Herausforderung beim Refactoring
- Probleme beim Merging

Gut für Apps mit einfacher und einheitlicher URL-Struktur

Attribute Routing

Pro

- Hohe Flexibilität
- Selbstdokumentierend
- Support für HTTP-Constraints

Contra

- Verstreute Konfiguration
- Potenzial für Duplizierung

Ideal für APIs mit komplexen und nicht einheitlichen URL-Patterns.

Conventional Routing

- `app.UseRouting()` um Routingfunktion zu aktivieren
- `app.MapControllers()` um attributgesteuerte Controller zuzuordnen

```
app.MapControllerRoute(  
    name: "default",  
    pattern:  
    "api/{controller}/{action}/{id?}");
```

Routing Attributes

<http://localhost:55387/Employee/All>

Response from GetAllEmployees Method

```
[ApiController]
public class EmployeeController : ControllerBase
{
    [Route("Employee/All")]
    [HttpGet]
    public string GetAllEmployees()
    {
        return "Response from GetAllEmployees Method";
    }
}
```

Routing Attributes mit Parametern

<http://localhost:55387/Employee/ById/420>

Response from GetEmployeeById Method Id: 420

```
[ApiController]
public class EmployeeController : ControllerBase
{
    [Route("Employee/ById/{Id}")]
    [HttpGet]
    public string GetEmployeeById(int Id)
    {
        return $"Response from GetEmployeeById Method Id: {Id}";
    }
}
```

Parameter von Query Strings

<http://localhost:55387/Employee/Search?City=London&Gender=Male&Department=HR>

Return Employees with Department: HR, Gender: Male, City: London

```
[ApiController]
public class EmployeeController : ControllerBase
{
    [Route("Employee/Search")]
    public string SearchEmployees([FromQuery]EmployeeSearch employeeSearch)
    {
        return $"Return Employees with Department: {employeeSearch.Department},
            Gender: {employeeSearch.Gender}, City: {employeeSearch.City}";
    }
}
```


Mehrere URLs für eine Ressource

```
[ApiController]
public class EmployeeController : ControllerBase
{
    [Route("Employee/All")]
    [Route("AllEmployees")]
    [Route("Employee/GetAll")]
    [HttpGet]
    public string GetAllEmployees()
    {
        return "Response from GetAllEmployees Method";
    }
}
```

Token Replacement

```
[ApiController]
[Route("[controller]")]
public class EmployeeController : ControllerBase
{
    [Route("[action]")]
    [HttpGet]
    public string GetAllEmployees()
    {
        return "Response from GetAllEmployees Method";
    }
    [Route("[action]")]
    [HttpGet]
    public string GetAllDepartment()
    {
        return "Response from GetAllDepartment Method";
    }
}
```

Token Replacement

```
[ApiController]
[Route("[controller]/[action]")]
public class EmployeeController : ControllerBase
{
    [HttpGet]
    public string GetAllEmployees()
    {
        return "Response from GetAllEmployees Method";
    }

    [Route("{Id}")]
    [HttpGet]
    public string GetEmployeeById(int Id)
    {
        return $"Response from GetEmployeeById Method, Id : {Id}";
    }
}
```

Route Constraints

```
[ApiController]
[Route("api/[controller]")]
public class EmployeeController : ControllerBase
{
    [Route("{EmployeeId:int:min(1000)}")] // nur Ganzzahlen ab 1000
    [HttpGet]
    public string GetEmployeeDetails(int EmployeeId)
    {
        return $"Response from GetEmployeeDetails, EmployeeId : {EmployeeId}";
    }
    [Route("{EmployeeName:alpha:minlength(5)}")] // nur Buchstaben, min. 5
    [HttpGet]
    public string GetEmployeeDetails(string EmployeeName)
    {
        return $"Response from GetEmployeeDetails, EmployeeName : {EmployeeName}";
    }
}
```

Model Binding

Web API – Model Binding

Mit verschiedenen Modellbindungstechniken können eingehende Requests wesentlich effizienter verarbeitet werden. Die Daten können aus verschiedenen Teilen eines HTTP-Requests extrahiert werden.

Mögliche Techniken

- Formulardaten [FormData]
- Query Strings [FromQuery]
- Route Data [FromRoute]
- HTTP-Header [FromHeader]
- Request Body [FromBody]

Typische Beispiele

Dateien hochladen oder Formulardaten empfangen

Typisch um Daten bei GET zu filtern

Typisch um eine ID zu übergeben

Authentication Token oder Metadaten übertragen

Typisch bei POST, PUT oder PATCH.

Content Type muss stimmen, (z. B. application/json)

Wie funktioniert das Model Binding in der WebAPI?

Das Framework versucht eingehende Daten basierend auf Datenquelle, Typ und den Attributen des Parameters an die Methoden zu binden. Der Prozess umfasst mehrere Schritte.

- **Bestimmung der Quelle:** Das Framework identifiziert zunächst die Quelle der (z. B. Route, Query, Body, Formular).
- **Bindung:** Basierend auf der Quelle versucht das Framework, die Daten an die Parameter der Methode zu binden. Komplexe Typen werden normalerweise mithilfe von Input Formatters aus dem Request gebunden, die die Daten in .NET-Typen deserialisieren.
- **Validierung:** Nach der Bindung erfolgt die Modellvalidierung. Wenn die Daten die Validierung nicht bestehen, enthält der Model-State die Validierungsfehler, die in der Action-Method überprüft werden können.

Best Practices

- **Richtige Source Attributes verwenden** mittels entsprechender Attribute eindeutig angeben. Dies verbessert die Lesbarkeit und Wartbarkeit Ihres Codes.
- **Validierung:** Sicherzustellen, dass die gebundenen Daten Ihren Kriterien entsprechen. ASP.NET Core unterstützt die automatische Modellvalidierung und kann HTTP 400-Antworten zurückgeben, wenn die Validierung fehlschlägt.
- **Spezifische Typen:** Modelle sollten der Datenstruktur der API bestmöglich entsprechen, um starke Typisierung und Validierung effektiv nutzen zu können.
- **Performance:** Größe und Komplexität der gebundenen Daten berücksichtigen, insbesondere bei großem JSON-Payload. Als Alternative evtl. Streaming oder Chunking verwenden.

Beispiel Formulardaten

```
<form action="https://localhost:55121/api/User" method="post">
  <input type="text" name="Name" placeholder="Name" required />
  <input type="email" name="Email" placeholder="Email" required />
  <button type="submit">Submit</button>
</form>
```

```
[Route("api/[controller]")]
[ApiController]
public class UserController : ControllerBase
{
    [HttpPost]
    public IActionResult CreateUser([FromForm] UserModel user)
    {
        // Handle the user data, e.g., save it to a database
        var response = new
        {
            Success = true,
            Message = $"User {user.Name} created successfully!",
            Code = StatusCodes.Status200OK
        };
        return Ok(response);
    }
}
```

```
public class UserModel
{
    public string Name { get; set; }
    public string Email { get; set; }
}
```

17

<https://dotnettutorials.net/lesson/fromform-in-asp-net-core-web-api/>

Ein Beispiel mit Formulardaten. Alle anderen Attribute funktionieren analog dazu.

Media Types

Web API – ActionResult-Types

- Einige Aktionsergebnistypen sind für ein bestimmtes Format spezifisch

- Wenn Native-Datentypen zurück gegeben werden, sind diese vom content-typ text/plain
- Einige ActionResult-Types sind für ein bestimmtes Format (content-typ) spezifisch:
- Formate (Mediatypes) im Überblick
 - text/plain
 - application/json
 - text/json

Ergebnisse von Aktionen, die Objekte zurückgeben, deren Typ nicht [ActionResult](#) ist, werden mit der entsprechenden [IOutputFormatter](#)-Implementierung serialisiert.

Return Types	Format	Beschreibung
NoContent	text/plain	HTTP Status-Code 204 wird zurück gegeben
JsonResult	application/json	Ergebnis wird als JSON zurück gegeben
ContentResult	text/plain	Einfache Zeichenketteausgabe

19

Ist ein RFC Standard

<https://tools.ietf.org/html/rfc2616>

Built-in Support: XML, JSON, BSON

<http://bsonspec.org/>

<https://dotnettutorials.net/lesson/controller-action-return-types-core-web-api/>

<https://docs.microsoft.com/en-us/aspnet/web-api/overview/formats-and-model-binding/content-negotiation>

<https://docs.microsoft.com/en-us/aspnet/web-api/overview/formats-and-model-binding/media-formatters>

Legacy .NET Framework

<https://www.tutorialsteacher.com/webapi/web-api-formatters>

Web API - Inhaltsaushandlung

- Eine Inhaltsaushandlung tritt auf, wenn der Client einen [Accept-Header](#) angibt
- Das von ASP.NET Core verwendete Standardformat ist JSON
- Für die Inhaltsaushandlung gilt:
 - Sie wird durch ObjectResult implementiert.
 - Sie ist in die Statuscode-spezifischen Aktionsergebnisse integriert, die von den Hilfsmethoden zurückgegeben werden. Die Hilfsmethoden für Aktionsergebnisse basieren auf ObjectResult
- Wenn ein Modelltyp zurückgegeben wird, lautet der Rückgabotyp -> ObjectResult
- Wenn das zurückgegebene Objekt null ist, wird eine 204 No Content-Antwort zurückgegeben.

Return Types	Beschreibung
Ok() / Ok(result)	Statuscode 200 + Rückgabeinhalt
NotFound() / NotFound(nameLike)	Boolean, gibt Auskunft darüber, ob der Formatter den Datentyp lesen kann.
POCO - Objects	Runtime erstellt automatisch ein ObjectResult
ReadFromStreamAsync	Wird beim Lesen des Input Streams aufgerufen. Gibt die Ressource zurück.
WriteToStreamAsync	Aufruf beim Schreiben der Ressource in den Output Stream.

20

Ist ein RFC Standard

<https://tools.ietf.org/html/rfc2616>

<https://docs.microsoft.com/en-us/aspnet/web-api/overview/formats-and-model-binding/content-negotiation>

<https://www.tutorialsteacher.com/webapi/web-api-formatters>

<https://docs.microsoft.com/en-us/aspnet/web-api/overview/formats-and-model-binding/media-formatters>

POCO - Plain old CLR object

Der Accept-Header

- Eine Aushandlung des Inhalts findet statt, wenn in der Anforderung ein Accept-Header angezeigt wird
 - Die Medientypen im Accept-Header werden in der bevorzugten Reihenfolge aufgelistet.
 - Es wird versucht, einen Formatter zu finden, der eine Antwort in einem der angegebenen Formate erzeugen kann.
 - In SwaggerUI kann man zum Testen einen Accept-Header mit angeben.

21

<https://www.tutorialsteacher.com/webapi/web-api-formatters>

JsonMediaTypeFormatter / XmlMediaTypeFormatter / FromUrlEncodedFormatter / JQueryMvcUrlEncodedMediaTypeFormatter / FormUrlEncodedMediaTypeFormatter erleben alle von MediaTypeFormatter.

MediaTypeFormatter-Klasse für die Verarbeitung von URL-endeten HTML-Formulardaten, auch als application / x-www-form-urlencoded bezeichnet.

Der Accept-Header

- Wenn kein Formatierer gefunden wird, der die Clientanforderung erfüllen kann, führt ASP.NET Core Folgendes aus:
 - Gibt 406 Not Acceptable zurück, MvcOptions.ReturnHttpNotAcceptable wenn auf festgelegt true ist
- ODER
- Es wird versucht, den ersten Formatter zu finden, der eine Antwort erzeugen kann.

22

<https://www.tutorialsteacher.com/webapi/web-api-formatters>

JsonMediaTypeFormatter / XmlMediaTypeFormatter / FromUrlEncodedFormatter / JQueryMvcUrlEncodedMediaTypeFormatter / FormUrlEncodedMediaTypeFormatter erleben alle von MediaTypeFormatter.

MediaTypeFormatter-Klasse für die Verarbeitung von URL-encodierten HTML-Formulardaten, auch als application / x-www-form-urlencoded bezeichnet.

Der Accept-Header

- Wenn kein Formatter für das angeforderte Format konfiguriert wurde, wird der erste Formatierer verwendet, der das Objekt formatieren kann:
 - Der erste Formatierer, der das Objekt verarbeiten kann, wird zum Serialisieren der Antwort verwendet.
 - Es findet keine Aushandlung statt. Der Server bestimmt, welches Format zurückgegeben werden soll.

23

<https://www.tutorialsteacher.com/webapi/web-api-formatters>

JsonMediaTypeFormatter / XmlMediaTypeFormatter / FromUrlEncodedFormatter / JQueryMvcUrlEncodedMediaTypeFormatter / FormUrlEncodedMediaTypeFormatter erleben alle von MediaTypeFormatter.

MediaTypeFormatter-Klasse für die Verarbeitung von URL-encodeten HTML-Formulardaten, auch als application / x-www-form-urlencoded bezeichnet.

Konfigurieren von Formatter

- Apps, die zusätzliche Formate unterstützen müssen, können die geeigneten NuGet-Pakete hinzufügen und die Unterstützung konfigurieren.
- Es gibt unterschiedliche Formatierungsprogramme für Ein- und für Ausgaben.
 - Eingabeformatierer werden vom Model-Binding verwendet.
 - Ausgabeformatierer werden zum Formatieren von Antworten verwendet.

24

IN-COURSE REMARKS

g

g

PREPARATION REMARKS

g

g

Hinzufügen von XML-Format Unterstützung

- Der XML-Formatter verwendet intern den XMLSerializer

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddControllers()
        .AddXmlSerializerFormatters();
}
```

25

IN-COURSE REMARKS

g

g

PREPARATION REMARKS

g

g

Konfigurieren System.Text.Json von basierten Formatter

- Features für die System.Text.Json basierten Formatter, können mithilfe von JsonSerializerOptions konfiguriert werden.

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddControllers()
        .AddJsonOptions(options => options.JsonSerializerOptions.PropertyNamingPolicy = null);
}
```

26

IN-COURSE REMARKS

g

g

PREPARATION REMARKS

g

g

Hinzufügen von Newtonsoft.Json

- Vor ASP.NET Core 3.0 wurden standardmäßig die JSON-Formatierer verwendet, die mithilfe des Newtonsoft.Json-Pakets implementiert wurden.
- Ab ASP.NET Core 3.0 basieren die Standardformatierer für JSON auf **System.Text.Json**.
- Unterstützung für Newtonsoft.Json basierte Formatierer und Features ist verfügbar
 - Installieren des Packages: Microsoft.AspNetCore.Mvc.NewtonsoftJson
 - Hinzufügen der Codezeile `.AddNewtonsoftJson();` in der Startup.cs -> ConfigureServices

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddControllers()
        .AddNewtonsoftJson();
}
```

27

IN-COURSE REMARKS

g
g

PREPARATION REMARKS

g
g

Wann verwende ich Newtonsoft.Json in ASP.NET Core

- Einige Features funktionieren mit System.Text.Json - Formatter nur mithilfe eines Verweises auf Newtonsoft.Json
- Verwenden Sie weiterhin Newtonsoft.Json-Formatter, wenn für die App Folgendes gilt:
 - Sie verwendet Newtonsoft.Json-Attribute. Zum Beispiel: [JsonProperty] oder [JsonIgnore]
 - Sie passt die Serialisierungseinstellungen an.
 - Sie nutzt Features, die von Newtonsoft.Json bereitgestellt werden.
 - WebApp wurde vor ASP.NET Core 3.0 erstellt und verwendet die **JsonSerializerSettings** Instanz.

28

IN-COURSE REMARKS

g

g

PREPARATION REMARKS

g

g

Konfigurieren System.Text.Json von basierten Formatter

- Funktionen für den Newtonsoft.Json-Formatter können mithilfe von Microsoft.AspNetCore.Mvc.MvcNewtonsoftJsonOptions.SerializerSettings konfiguriert werden:

```
services.AddControllers().AddNewtonsoftJson(options =>
{
    // Use the default property (Pascal) casing
    options.SerializerSettings.ContractResolver = new DefaultContractResolver();

    // Configure a custom converter
    options.SerializerSettings.Converters.Add(item: new MyCustomJsonConverter());
});
```

- Optionen zur Ausgabeserialisierung können aktionsweise mithilfe von JsonResult konfiguriert werden. Beispiel:

```
public IActionResult Get()
{
    return Json(model, new JsonSerializerSettings
    {
        Formatting = Formatting.Indented,
    });
}
```

29

IN-COURSE REMARKS

g
g

PREPARATION REMARKS

g
g

Angeben eines Formats

- Wenden Sie den Filter an, um die Antwortformate [Produces] einzuschränken.
- Wie die meisten Filter, kann [Produces] auf die Aktion, den Controller oder den globalen Bereich angewendet werden:

```
[ApiController]
[Route(template: "[controller]")]
[Produces(contentType: "application/json")]
3 references
public class WeatherForecastController : ControllerBase
{
    private static readonly string[] Summaries = new[]
    {
        "Freezing", "Bracing", "Chilly", "Cool", "Mild", "Warm", "Balmy", "Hot", "Sweltering", "Scorching"
    };
}
```

30

IN-COURSE REMARKS

g

g

PREPARATION REMARKS

g

g

Lab - Time: Formatter für besondere Fälle + Zuordnung des Antwortformats durch URLs

Lab - Time: Benutzerdefinierten Formatter

[Benutzerdefinierte Formatierer in Web-APIs in ASP.NET Core | Microsoft Docs](#)